SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. *(BL3)*

Assessment Tool Weightage: 50%

QUESTIONS: 5 **Duration : 1 Hour**
Total Marks:50

Annexure A

Analytical Problem Solving

Code of Conduct:

*I **MATHIMOZHI ER - 192321121** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Analytical Problem Solving

1. Solve the Water Jug problem: you are given 2 jugs, a 4-gallon one and 3-gallon one. Neither has any measuring maker on it. There is a pump that can be used to fill the jugs with water. How can you get exactly 2 gallons of water into 4-gallon jug? Explicit assumptions: A jug can be filled from the pump, water can be poured out of a jug onto the ground, water can be poured from one jug to another and that there are no other measuring devices available.

Answer:

1. Problem formulation (state-space search)

State: an ordered pair (x, y), where x = water in the 4-gallon jug (0 to 4) and y = water in the 3-gallon jug (0 to 3).

Initial state: (0, 0), both jugs empty.

Goal test: x = 2, i.e. any state (2, y). The question asks for exactly 2 gallons in the 4-gallon jug.

Path cost: 1 per action, so the best solution is the one with the fewest steps.

2. Operators (production rules)

No.	Condition	Action	New state
1	$x < 4$	Fill the 4-gallon jug from the pump	(4, y)
2	$y < 3$	Fill the 3-gallon jug from the pump	(x, 3)
3	$x > 0$	Empty the 4-gallon jug on the ground	(0, y)
4	$y > 0$	Empty the 3-gallon jug on the ground	(x, 0)
5	$x + y \geq 4$ and $y > 0$	Pour 3-jug into 4-jug until the 4-jug is full	$(4, y - (4 - x))$
6	$x + y \geq 3$ and $x > 0$	Pour 4-jug into 3-jug until the 3-jug is full	$(x - (3 - y), 3)$
7	$x + y \leq 4$ and $y > 0$	Pour all of the 3-jug into the 4-jug	$(x + y, 0)$
8	$x + y \leq 3$ and $x > 0$	Pour all of the 4-jug into the 3-jug	$(0, x + y)$

3. Solution (applying the rules step by step)

Step	Rule used	State (x, y)	What happens
0	–	(0, 0)	Start: both jugs empty
1	Rule 2	(0, 3)	Fill the 3-gallon jug from the pump
2	Rule 7	(3, 0)	Pour all 3 gallons into the 4-gallon jug
3	Rule 2	(3, 3)	Fill the 3-gallon jug again
4	Rule 5	(4, 2)	Pour from the 3-jug into the 4-jug. The 4-jug needs only 1 more gallon, so 2 gallons stay in the 3-jug
5	Rule 3	(0, 2)	Empty the 4-gallon jug on the ground
6	Rule 7	**(2, 0)**	Pour the 2 gallons from the 3-jug into the 4-jug. **GOAL reached**

Fig. 1: State-space path for the Water Jug problem, (0,0) to (2,0) in 6 steps

4. Example and result

Example check of step 4: the state is (3, 3), so the 4-jug has 3 gallons and the 3-jug has 3 gallons. Pouring the 3-jug into the 4-jug fills only 1 gallon (4 − 3 = 1), so the 3-jug is left with 3 − 1 = **2 gallons**. Emptying the 4-jug and pouring those 2 gallons across puts exactly **2 gallons in the 4-gallon jug**.

Alternative 6-step solution (also optimal): (0,0) → (4,0) → (1,3) → (1,0) → (0,1) → (4,1) → (2,3). Breadth-First Search on this state space shows that no solution with fewer than 6 steps exists, so both answers are optimal.

Final answer: the 4-gallon jug contains exactly 2 gallons after 6 operations.

2. Find out how Mars Rover, analyse and explore the samples on Mars by answering the following questions.
   i. What are the percept's for this agent?

Answer:

A percept is whatever the rover senses at one instant. The Mars Rover receives the following percepts:

Sensor	Percept received	Used for
Cameras (mast / navigation)	Images of terrain, rocks, colour, shadows	Path planning, picking interesting samples
Spectrometer / chemical sensors	Mineral and chemical readings of rock or soil	Analysing samples (water, carbon, minerals)
Wheel encoders, gyroscope, accelerometer	Distance travelled, tilt angle, wheel slip	Navigation and avoiding toppling
Temperature and dust sensors	Ambient and equipment temperature, dust level	Protecting electronics
Battery / solar sensors	Charge level, power generated	Energy management
Radio receiver	Commands and updates from Earth	Mission goals and new instructions

Example percept: [camera: large rock 2 m ahead, tilt: 12°, battery: 40%, spectrometer: sample shows iron-rich mineral].

   ii. Characterize the operating environment.

Answer:

Environment characterisation (PEAS "E" and task-environment properties):

Property	Mars Rover environment	Reason
Observable	Partially observable	Cameras see only a limited range; the ground under a rock or inside soil is hidden
Agents	Single agent	Only the rover acts (Earth control gives goals, not moves)
Deterministic?	Stochastic	Wheels may slip on sand, dust storms and falling rocks cause unpredictable results
Episodic?	Sequential	Today's move (battery use, position) affects later decisions
Static?	Dynamic	Weather, dust and light change while the rover is thinking
Discrete?	Continuous	Position, speed, temperature and time take continuous values
Known?	Largely unknown	The terrain is new and unexplored, so the rover has to learn it

So the rover works in a **partially observable, stochastic, sequential, dynamic, continuous, single-agent** environment: one of the hardest types for an agent.

   iii. What are the actions the agent can take?

Answer:

The rover's actuators allow these actions:

- **Movement:** move forward / backward, turn left / right, stop, climb or avoid an obstacle, change speed.
- **Sampling:** extend the robotic arm, scoop soil, drill into rock, place the sample into the analyser.
- **Sensing:** take a photograph, rotate the camera mast, run the spectrometer.
- **Communication:** transmit data to the orbiter / Earth, receive commands.
- **Self-maintenance:** open solar panels, charge, switch to sleep (low-power) mode, heat the electronics.

Example: if percept = "large rock ahead, 2 m", the rover chooses the action TURN-LEFT 30° then MOVE-FORWARD.

iv. How can one evaluate the performance of the agent?

Answer:

The performance measure checks how well the rover is achieving its mission:

Criterion	How it is measured	Effect
Science value	Number and quality of useful samples analysed	Higher is better (+)
Exploration	Area covered / number of target sites reached	Higher is better (+)
Safety	Collisions, tilt-overs, damage events	Lower is better (−)
Energy efficiency	Battery used per task, no power failure	Lower use is better (−)
Data return	Amount of correct data sent to Earth	Higher is better (+)
Lifetime	Number of sols (Mars days) operating	Higher is better (+)

One simple scoring function is: **Performance = w1·(useful samples) + w2·(area explored) + w3·(data sent) − w4·(damage) − w5·(energy wasted)**, where w1…w5 are importance weights set by the mission team.

v. What sort of agent architecture do you think is most suitable for this agent? Why?

Answer:

The most suitable design is a **utility-based agent** (with a world model and goals, and a learning element on top).

Why the other types are not enough

- **Simple reflex agent:** reacts only to the current percept. It would fail because Mars is partially observable (it forgets the map, the battery trend and where it has been).
- **Model-based reflex agent:** remembers the state, but has no mission target, so it does not know what to aim for.
- **Goal-based agent:** can pursue "reach site X and drill", but cannot compare competing goals (a risky rock with high science value versus a safe rock with low value).

Why a utility-based agent fits

- It keeps an **internal model** of the world (map, soil, battery), which helps in a partially observable environment.
- It has **goals** (reach target, collect sample).
- Its **utility function** weighs science value against risk and energy, so it can handle trade-offs and uncertain (stochastic) outcomes by maximising expected utility.
- A **learning element** improves terrain judgement over time (for example, which surfaces cause slipping).

Fig. 2: Utility-based agent architecture for the Mars Rover

PEAS	Mars Rover
Performance	Samples analysed, area explored, safety, energy efficiency, data returned
Environment	Mars surface: rocks, soil, slopes, dust, temperature, radio delay
Actuators	Wheels, robotic arm, drill, sample analyser, camera mast, antenna
Sensors	Cameras, spectrometer, gyro, encoders, temperature, battery, radio

3. Explore the search space using search strategies and formulate the problem components for the 8-Queens problem using the following information: Place 8 queens on a chessboard such that none of the queen's attack any of the others. A configuration of 8 queens on the board as shown in the figure below, but this does not represent a solution as the queen in the first column is on the same diagonal as the queen in the last column.

Answer:

1. Problem formulation (incremental formulation)

Component	8-Queens definition
States	Any arrangement of 0 to 8 queens on the board. Better version: one queen in each of the leftmost k columns with no two queens attacking each other.
Initial state	Empty board (no queens placed)
Actions	Add a queen to the next empty column, in any row that is not attacked by queens already placed
Transition model	Returns the board with the new queen added
Goal test	8 queens on the board and no queen attacks another (no shared row, column or diagonal)
Path cost	Not important: only the final arrangement matters (every action costs 1)

2. Size of the search space

Formulation	States to explore
Complete-state: place all 8 queens anywhere	$64C8 \approx 4.4 \times 10^9$ arrangements
One queen per column, any row	$8^8 = 16{,}777{,}216$
One queen per column and per row (permutations)	$8! = 40{,}320$
Incremental, never place an attacked queen	only 2,057 states (including the empty board)

The smart formulation reduces the search space from billions of states to about two thousand. The 8-Queens problem has **92 solutions** (12 distinct ones if rotations and reflections are treated as equal).

3. Search strategy and example

Every solution lies at depth 8, and path cost is irrelevant, so **Depth-First Search with backtracking** is the right strategy: it uses little memory (one path only) and reaches the goal depth quickly. BFS would store millions of partial boards.

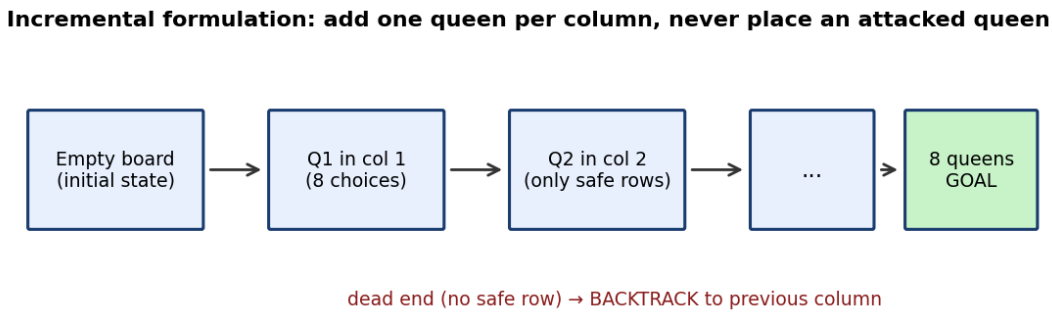


Fig. 3: Incremental search with backtracking

Example trace (rows numbered 1 to 8 from the bottom): place Q1 in column 1, row 1. In column 2, row 1 (same row) and row 2 (diagonal) are attacked, so the first safe row is 3. The search continues column by column, and when a column has no safe row (a dead end) it backtracks and moves the previous queen to its next safe row. DFS reaches the first solution after placing only 113 queens in total.

Fig. 4: Left: failing configuration (Q1 and Q8 on one diagonal). Right: a solution

Reading the figure: on the left, the queen in column 1 (row 1) and the queen in column 8 (row 8) lie on the same diagonal, so this board fails the goal test, exactly as the question says. The right board is a valid goal state: the rows 1, 5, 8, 6, 3, 7, 2, 4 (column 1 to 8) are all different and no two queens share a diagonal.

Diagonal check used in the goal test: two queens at (row r1, column c1) and (row r2, column c2) attack each other if $r1 = r2$ or $|r1 - r2| = |c1 - c2|$.

4. A customer wants to travel from the one location to another location using OLA Cab booking mobile application. The customer can select the any of the cab type as mini, micro, sedan, shared, prime and so on based on his comfort to reach the destination. Identify what type of problem-solving agent can be used and write the pseudocode for the above problem.

Answer:

1. Type of agent

The OLA application should be a **goal-based problem-solving agent**. It has a clear goal (reach the destination), formulates a problem from the pickup location, drop location and the available cab types, searches for the best option and executes the booking. Because the customer chooses the cab type on comfort, fare and waiting time, the choice step uses a **utility function**, so the agent is a goal-based agent with a utility-based selection step.

2. Problem formulation

Component	OLA problem
Initial state	Customer at pickup location, no cab booked
Goal state	Customer reaches the destination
Actions	Choose cab type (Mini, Micro, Sedan, Shared, Prime…), confirm booking, follow route, end trip
Transition model	Booking confirmed → cab arrives → trip in progress → destination reached
Path cost	Fare + travel time (+ waiting time), adjusted by customer comfort

3. Pseudocode

```
function OLA-AGENT(percept) returns an action
    persistent: seq, state, goal, problem
```

```
    state <- UPDATE-STATE(state, percept)
    if seq is empty then
        goal    <- FORMULATE-GOAL(state)        // reach the destination
        problem <- FORMULATE-PROBLEM(state, goal)
        seq     <- SEARCH(problem)              // route + cab type
        if seq = failure then return NULL-ACTION
    action <- FIRST(seq);  seq <- REST(seq)
    return action

function BOOK-RIDE(pickup, drop, preference)
    cabs <- {MINI, MICRO, SEDAN, SHARED, PRIME}
    available <- []
    for each c in cabs do
        if AVAILABLE(c, pickup) then add c to available
    if available is empty then return "No cabs available"
    for each c in available do
        fare[c] <- ESTIMATE-FARE(c, pickup, drop)
        eta[c]  <- ESTIMATE-ARRIVAL-TIME(c, pickup)
        U[c]    <- w1*COMFORT(c) - w2*fare[c] - w3*eta[c]
    best <- the cab c in available with maximum U[c]
    show options to customer; customer confirms (default = best)
    BOOK(chosen); TRACK(chosen) until pickup
    while destination not reached do  FOLLOW-ROUTE()
    END-TRIP();  COLLECT-PAYMENT()
    return "Ride completed"
```

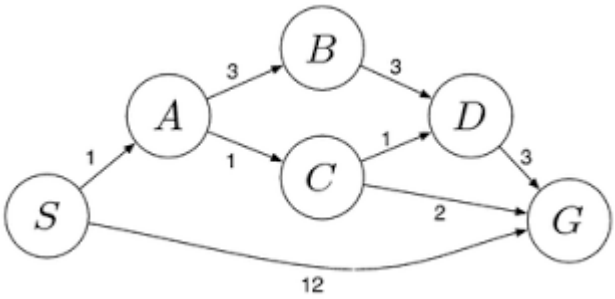
4. Example

Customer travels from Chennai Central to Chennai Airport and prefers comfort. With w1 = 20, w2 = 0.1, w3 = 1 and **illustrative** fares:

Cab type	Comfort (1-5)	Fare (₹)	Arrival (min)	Utility U
Shared	1	190	12	−11
Micro	2	280	3	9
Mini	3	320	4	24
Sedan	4	400	5	35
Prime	**5**	**450**	**6**	**49**

The agent suggests **Prime** because it has the highest utility for this customer. A budget customer with a higher w2 would get Mini or Shared. The goal (reach the destination) stays the same; only the utility changes the chosen cab.

5. A logistics company operates a delivery network where packages must be transported between warehouses at minimum cost. Each route between warehouses has an associated travel cost (fuel + time). The company wants to determine the least-cost path from the starting warehouse **S** to the destination warehouse G using the Uniform Cost Search (UCS) algorithm.
   The delivery network is represented as a weighted graph:



Answer:

1. Problem understanding

Edges of the weighted graph (read from the figure): S–A = 1, S–G = 12, A–B = 3, A–C = 1, B–D = 3, C–D = 1, C–G = 2, D–G = 3. Start = S, Goal = G. **Uniform Cost Search** always expands the frontier node with the lowest path cost g(n) from S, using a priority queue. Since it never expands a costlier node first, the first time G is removed from the queue its path is the cheapest.

2. Step-by-step execution

Iter.	Node expanded (g)	Children generated (cost from S)	Frontier after step (sorted by g)
1	S (0)	A = 1, G = 12	A(1), G(12)
2	A (1)	B = 1+3 = 4, C = 1+1 = 2	C(2), B(4), G(12)
3	C (2)	D = 2+1 = 3, G = 2+2 = 4 (better than 12, replaces it)	D(3), B(4), G(4)
4	D (3)	G via D = 3+3 = 6, worse than 4, discarded	B(4), G(4)
5	B (4)	D via B = 4+3 = 7, worse than 3, discarded	G(4)
6	G (4)	**Goal test passes, STOP**	–

3. Result

Path from S to G	Total cost
S → A → C → G (optimal)	**1 + 1 + 2 = 4**
S → A → C → D → G	1 + 1 + 1 + 3 = 6
S → A → B → D → G	1 + 3 + 3 + 3 = 10
S → G (direct)	12

4. Interpretation

The least-cost route is **S → A → C → G with a total cost of 4** cost units (fuel + time). The direct road S → G looks shortest on the map, but it costs 12, so the company saves 8 units (about 67%) by going through warehouses A and C. If G and B tie at cost 4, the answer does not change because both have cost 4 and G is the goal. UCS is **complete** (all edge costs are positive) and **optimal**, so no cheaper route exists.

5. Pseudocode

```
function UCS(graph, S, G)
    frontier <- priority queue ordered by g;  insert (S, g = 0)
    explored <- empty set
    loop
        if frontier is empty then return failure
        node <- POP lowest-g node from frontier
        if node = G then return PATH(node), g(node)
        add node to explored
        for each neighbour n of node with edge cost c do
            g_new <- g(node) + c
            if n not in explored and not in frontier then insert (n, g_new)
            else if n in frontier with larger g then replace it by (n, g_new)
```

Final answer: optimal path = S → A → C → G, minimum cost = 4.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem